程浚航

13224950545 · chengjunhang7@gmail.com · https://cjhcoder7.github.io/ · GitHub @cjhCoder7

教育背景

北京航空航天大学, 计算机科学与技术, 本科 2023.9 - 2027.6

北京航空航天大学, 计算机科学与技术, 博士 （预计）2027.9 - 2031.6

实习经历

腾讯混元基础模型部, Agent 前沿评测算法实习生（青云计划） 2026.11 - 至今

- 围绕 Coding Agent 能力评测，关注任务设计、执行结果分析与失败案例归因。

虾皮 LLM Team, 大模型预训练算法实习生 2026.8 - 2026.10

- 参与虾皮 Compass 模型代码预训练数据建设，负责数据处理管线、分类器训练与数据定向合成，**推动内部 35B-A3B 模型在 HumanEval、MBPP 等常见预训练代码生成评测上持平甚至超过 Qwen3.5 35B Base**。
- Code 预训练数据处理管线
  - 质量、多样性与语言分类器：针对开源代码数据质量参差、缺少领域与语言标注的问题，**以 LLM 标注为监督信号，基于 FastText 训练三类分类器**。质量分类器将 LLM 的 1-5 分标注转为二分类（1、2 分为负例，4、5 分为正例）；多样性分类器覆盖代码领域与功能；语言分类器为数据补充编程语言标签，30w 条标注数据训练后，**三种分类器精确率在 90% 左右，召回率在 80% 左右**。
  - 未定义变量筛选：针对低质量样本直接送 LLM 打标成本过高的问题，面向 Python、Rust、C++、Java 等十种语言，**基于编译器前端构建未定义变量与批量导入检测流程**，前置剔除问题样本，**显著降低下游 LLM 打标成本**。
- Code 预训练数据定向合成：针对模型在 HumanEval、MBPP 上能力不均衡的问题，**将 Benchmark 拆解为字符串操作等细粒度子能力**以定位短板，**针对薄弱子能力定向合成代码数据**。

Human Intelligence, Terminal 数据实习生 2026.7

- TB 题目机审优化：分析人工提交 TB 题目的常见拒绝原因，据此优化机审 prompt，**有效降低机审成本**。
- TB 数据合成：针对人工构造 TB 任务效率低的问题，构建 **Skills 知识图谱注入 + 多领域场景注入 + 人工 Seed 题目**的合成方案，所合成任务大多可使 **Opus 4.6、GPT 5.5 求解步数达到 20 步以上**。
- CoT 提取：针对闭源模型不开放完整思维链的问题，落地适配**思维链重放与 Think Tag 注入**两种方式，**能稳定实现 Opus 4.6 - Opus 4.8 的 CoT 提取**。
- Game-TerminalBench：针对 Agentic 任务离散 Reward 导致 RL 奖励稀疏、既有游戏任务依赖 VLM 的问题，**探索基于游戏游玩场景的新型 TB Benchmark**，以 CLI 工具交互并用文本反馈游戏状态，**兼顾 Long-Horizon 特性与纯文本 LLM 评测**；**共合成 76 个任务，DeepSeek V4 Flash 平均解题步数约 1000 步**。

九坤 IQuest Lab, 大模型后训练算法实习生 2026.1 - 2026.6

- 参与 IQuest-Coder 基座模型研发，**负责 SWE 方向后训练数据的全链路建设**，覆盖仓库筛选、环境构建、Instance 合成、轨迹蒸馏与任务评测，**累计构建约 20 万 SWE Instance、270 万条 SWE 轨迹**，提升模型在 SWE-Bench Verified、SWE-bench Multilingual 上的任务解决能力，**IQuest-Q1 Contributor**。
- **任务合成**
  - Repo 筛选与标注
    - 综合 Star 数、Commit 活跃度、许可证、编程语言、是否 Fork、README 质量等维度初筛开源仓库，并完成内部去重与 Benchmark 去重。
    - 按**编程语言、应用领域、应用功能三套体系**标注仓库，建立 Repo 级数据库，并**固化为定期更新的自动化流水线**。
  - 环境构建
    - 早期 SWE 任务以 Python 为主，采用 Conda 环境模板结合 LLM Workflow 构造 Dockerfile 环境。
    - 针对固定模板难以扩展多语言的问题，升级为**基于编程语言 Skills 与 Agent 的语言无关环境合成方法，在扩展多语言的同时提升环境合成准确率**，环境构建成功率在 40% 左右。
  - Instance 合成：所有 Instance 均经 **F2P/P2P 验证与多次运行去 flaky** 后入库
    - 基于 Issue 的题目合成：从 Issue 及关联 PR 合成题目，过滤 issue 文本中包含 patch 片段的样本，要求同时包含 Golden Patch 与 Test Patch 并验证可解，**保证题目可执行、可验证**。
    - 基于语法树的 Bug 合成：参考 SWE-Smith，**基于 Tree-sitter 通过函数重写、改写与规则化改写注入 Bug**，支持多 Bug 组合，这一类 Instance 用于后续大规模产出 Mid-training Agentic 数据。

    - 基于语法树的新特性增加：结合测试运行情况定位源码功能簇并**挖空构造新特性增加类题目**，对齐 DeepSWE 等新型 SWE 中 diff 行数较多的任务。
    - Instance 改写：对优质 Instance 的 instruction **改写为信息不全、贴近真实 Agent 使用场景的多个版本**，缩小与实际用户使用 prompt 的差距。
- **轨迹蒸馏**：将内部的类 OpenHands 基建**迁移至 Harbor 并支持离线安装**，并接入 **Claude Code（LiteLLM）**、**Codex（LiteLLM）与 Terminus2** 等框架，统一调度蒸馏任务，**每题 5-10 次 rollout**，同时使用 Avg@5 作为 Instance 难度标记，**reward 过滤后累计产出约 270 万条求解轨迹**。
- **训练数据清洗与训练**
  - 轨迹数据筛选：将轨迹产出的 patch 回放 FAIL_TO_PASS 与 PASS_TO_PASS 测试，**仅保留执行验证通过的轨迹**；规则化过滤重复命令、死循环、连续工具报错、超 turn 上限以及查看 git log、篡改测试等**异常与作弊行为**；按 repo、语言与任务难度配平并限制单题保留数量，保证多样性。
  - 模型训练与效果：针对轨迹中低质量 turn 拖累训练效果的问题，**开发一套 rubric 规则识别低质量 turn**（错误工具调用、过长 thinking 等）并**对其做 loss mask**，消融实验**可带来约 3 pt 提升**；最终在内部 30B-A3B ckpt 的 SFT 上，**多个 Harness 下 SWE-bench Verified 与 SWE-bench Multilingual 均达到 77+ 的水平**。
- **任务评测**
  - Hack 防范：从环境、网络与轨迹三层构建防护：**离线沙箱隔离**；**allowlist 网络白名单**仅放行模型服务与特定站点；禁止 git log 等命令并**实时清除轨迹中的 Hack Step**。
  - 评测协议：**pass@1 取 5 次运行平均**，单任务 Agent 运行上限 2 小时，保证结果稳定可比。
  - 环境与题目修复：针对 Java 等仓库运行时才拉取依赖导致环境不完整的问题，在镜像构建阶段**预先运行测试以预热依赖缓存**，实现离线评测；针对部分 Instance **Issue 描述不清晰、难以推出解法**的问题，补全关键信息并重写描述，保证题目可解。

论文工作

- **CangjieBench: Can LLM Coding Assistants Support Emerging Programming Languages? An Empirical Study on Cangjie** (*ESEM*, 2026)
  Junhang Cheng, Fang Liu, Jia Li, Chengru Wu, Nanxiang Jiang, Li Zhang.
- **AdaptiveLLM: A Framework for Selecting Optimal Cost-Efficient LLM for Code-Generation Based on CoT Length** (*Internetware*, 2025)
  Junhang Cheng, Fang Liu, Chengru Wu, Li Zhang.
- **VeriScope: Measuring Verification-Ready Verilog Artifacts** (*NeurIPS Datasets and Benchmarks Track*, 2026)
  Wei Zhang, Jian Yang, Jiajun Wu, Junhang Cheng, Chufan He, Yihang Lou, Xianglong Liu.
- **SWE-Crafter: Scaling Executable Multilingual Software Engineering Data with Meta-Skill Agents** (*NeurIPS*, 2026)
  Shukai Liu, Jian Yang, Bo Jiang, Junhang Cheng, Haowen Wang, Xiao-Hui Li, Xianglong Liu.
- **InCoder-32B-Thinking: Industrial Code World Model for Thinking** (*EMNLP*, 2026)
  Jian Yang, Wei Zhang, Jiajun Wu, Junhang Cheng, Tuney Zheng, Fanglin Xu, Weicheng Gu, et al.
- **InCoder-32B: Code Foundation Model for Industrial Scenarios** (*arXiv*, 2026)
  Jian Yang, Wei Zhang, Jiajun Wu, Junhang Cheng, Shawn Guo, Haowen Wang, Weicheng Gu, et al.
- **Towards Realistic Project-Level Code Generation via Multi-Agent Collaboration and Semantic Architecture Modeling** (*TOSEM*, 2026)
  Qianhui Zhao, Li Zhang, Fang Liu, Junhang Cheng, Chengru Wu, Junchen Ai, Qiaoyuanhe Meng, et al.

竞赛获奖

- 2026 年：华为奖学金。
- 2025 年：全国“挑战杯”主赛道**一等奖**，首都“挑战杯”主赛道**特等奖**，北航“冯如杯”主赛道制作组**二等奖**，沈元学院“万新杯”创新赛道**特等奖**。
- 2025 年：美国大学生数学建模竞赛 (MCM) **Meritorious Winner**；北航学科竞赛奖学金**特等奖**，国家励志奖学金，北航学习优秀奖学金二等奖。
- 2024 年：北航学习优秀奖学金二等奖。